Singapore Rainfall Analysis and Prediction



Spark-based weather-pattern analysis and 30-minute rainfall probability prediction

Scalable Distributed Computing for Data Science DSA5208

AUTHOR	STUDENT ID
Amrit Satpathy	A0355181E
Ritik Srivastava	A0355860X

Dataset: National Environment Agency, Historical Rainfall across Singapore · data.gov.sg

October 2026

Contents

Abstract....3
1. Introduction and objectives....4
2. Dataset and data preparation....4
3. Computing architecture....5
4. Exploratory data analysis....6
5. Task 2 — rainfall-probability prediction....10
6. Model and compute experiments....13
7. Threats to validity and proposed extensions....16
8. Limitations and conclusion....16
9. AI usage declaration....17
References....17
Appendix A. Distributed systems architecture....18
Appendix B. Model architecture atlas....21

Abstract

This project uses Apache Spark to analyse 48,538,713 cleaned five-minute rainfall readings from 91 monitoring stations across Singapore between 2017 and 2024. It addresses two tasks: describing seasonal, diurnal, interannual and spatial rainfall patterns, and estimating the probability that a station will accumulate more than a user-selected amount of rainfall in the next 30 minutes.

The rainfall summaries show pronounced variation. Mean annual rainfall per station is lowest in 2019 (1,526 mm) and highest in 2022 (2,750 mm). Wet readings peak around 14:00–16:00 Singapore Time. The annual station inventory grows from 62 devices in 2017 to 70 in 2024, while the H3 and official-region views show persistent spatial heterogeneity rather than a uniform island-wide pattern.

The submitted Spark MLlib Gradient-Boosted Trees pipeline is linked to a saved-model record with 7,997,496 threshold-expanded training rows and 31,872,380 2024 test rows. It reports ROC-AUC 0.924 and PR-AUC 0.584. The reported F1 of 0.984 is MLlib's weighted F1; the retained evaluation artifacts do not establish positive-class precision, recall, F1 or probability calibration. Operational rare-event claims are therefore outside the supported scope.

> **Systems observation.** On the shared laptop, local[*] completed the reported workloads sooner than the tested Standalone and Docker layouts. This is consistent with process, serialisation and container overhead on a single host; it should not be read as evidence against multi-machine Spark scaling.

1. Introduction and objectives

Rainfall at five-minute resolution is temporally clustered: consecutive wet readings can represent the development and passage of a shower, while most station-time observations are dry. This makes recent rainfall, local time and station context useful for short-horizon probability prediction.

1.1 Project tasks

- Task 1 — explore how rainfall changes within a year and across 2017–2024, using appropriate summaries and visualisations.
- Task 2 — estimate P(next-30-minute accumulation > q) for a specified station, prediction time and rainfall threshold q using Spark MLlib.
- Additional study — compare model quality and deployment cost while describing the limits of a single-host distributed experiment.

1.2 Scope and interpretation

The observations come from monitoring stations rather than one island-wide gauge. Station availability changes over time, so per-station normalisation improves comparison but does not fully remove coverage bias. The public source also warns that missing observations can occur and that the data have not undergone the same quality-control procedures as official climate records [1].

2. Dataset and data preparation

Table 1. Dataset scope and retained data profile.

Item	Reported value / treatment
Years	2017–2024 inclusive
Raw size	Approximately 7.8 GB across eight CSV files
Cleaned records	48,538,713
Distinct stations	91
Storage	Parquet partitioned by year
Time convention	Asia/Singapore

2.1 Cleaning rules

- Parse timestamps under the Asia/Singapore Spark session time zone and cast rainfall and coordinates to numeric types.
- Remove records with missing station/time/rainfall, negative rainfall, or rainfall above the project screening bound of 60 mm per five minutes.
- Deduplicate (station_id, timestamp), retaining the record with the latest update timestamp.
- Resolve a canonical station name and median coordinates for each station ID.
- Write the cleaned columns to Snappy-compressed Parquet partitioned by year.

Data-quality note. The 60 mm screening value is a project rule rather than a documented sensor limit. The retained artifacts do not provide rule-level removal counts, cadence-gap totals, sensitivity to this bound or a trace audit of the reported 786 mm station-day in 2022. These omissions limit data-quality inference but do not alter the descriptive status of the reported summaries.

3. Computing architecture

The architecture separates distributed data preparation from model fitting. Spark performs ingestion, validation, aggregation, per-station temporal windows and MLlib training. Small summaries are collected only for plotting. The PyTorch branch receives a bounded sequence sample and runs on one laptop GPU; it is supplementary and is not distributed training. Figures A2–A6 document the logical data plane and the three execution topologies in detail.

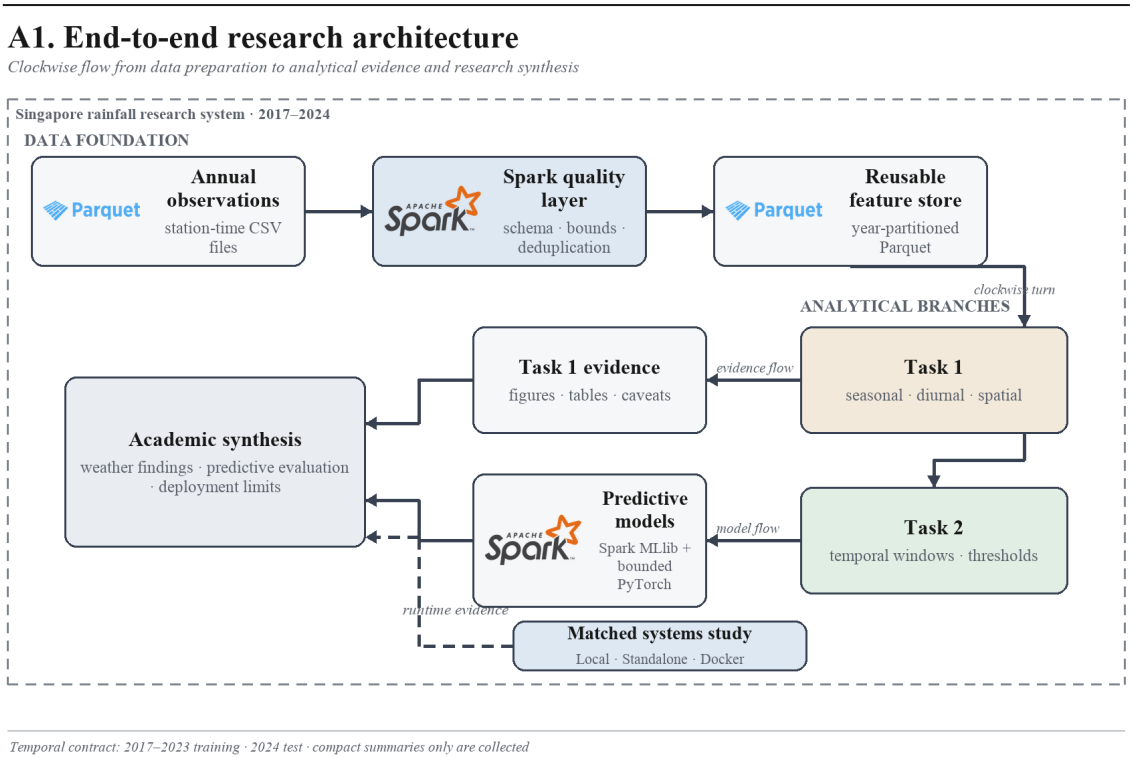


Figure 1. End-to-end analytical workflow from annual rainfall files to descriptive evidence, predictive models and systems measurements.

3.1 Environment and resource configurations

Table 2. Spark configurations and requested resources; settings are not verified runtime allocations.

Configuration	Execution	Requested resources	Shuffle partitions
local-all	local[*], one JVM	16 host threads; 20 GB driver heap	96
standalone-2x2	2 executor processes	4 total cores; 4 GB driver; 2 × 2 GB executors	32
standalone-4x2	4 executor processes	8 total cores; 4 GB driver; 4 × 2 GB executors	64
docker-2w	2 worker containers	4 total cores; 2 GB driver; 2 × 1 GB executors	32
docker-3w	3 worker containers	6 total cores; 2 GB driver; 3 × 1 GB executors	48

Host specification: Windows laptop with WSL2 Ubuntu, 32 GB physical RAM, 24 GB exposed to WSL2, 16 logical CPU cores and 8 GB swap. Software: Python 3.12, OpenJDK 21 and PySpark 3.5.3. Docker workers advertise two cores and 3 GB each; Compose does not impose hard CPU or memory limits. Requested settings should therefore be distinguished from realised runtime allocation.

Meaning of "distributed". Standalone and Docker use separate Spark executor processes and network shuffle, but do not add physical CPUs, memory channels, storage or network interfaces. They demonstrate deployment mechanics and overhead, not multi-node scalability.

4. Exploratory data analysis

Task 1 examines spatial, interannual, seasonal and diurnal rainfall variation. Spark performs the large aggregations, while only compact summaries are collected for plotting. Station and monthly values are normalised before comparison so that changes in the observing network do not directly determine the visual pattern. The resulting figures describe the monitored network rather than a spatially complete rainfall climatology.

4.1 Geospatial exploratory analysis

Spatial variation is examined at three complementary scales: station coverage, sensor-supported H3 cells and the five official Master Plan 2019 planning regions. Monthly rainfall is coverage-normalised before mapping. H3 polygons appear only where station evidence is available; no rainfall surface is interpolated across ungauged land.

4.1.1 Observation network

(a) 2017 **(b) 2024**

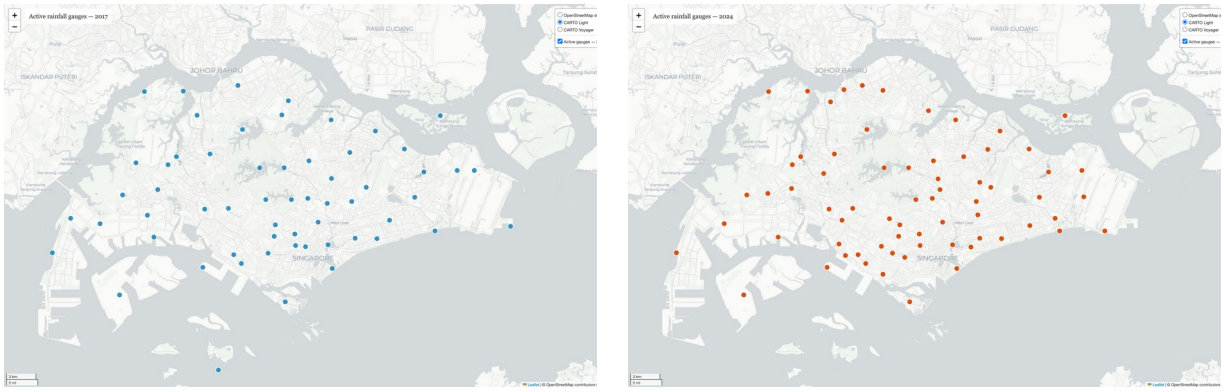


Figure 2. Active rainfall gauges in 2017 (left) and 2024 (right).

The annual inventory increases from 62 devices in 2017 to 70 in 2024. The comparison also shows why raw event counts need a coverage denominator: the number and location of operating gauges change over time, and several devices report for only part of a year.

4.1.2 August comparison

(a) August 2017 **(b) August 2024**

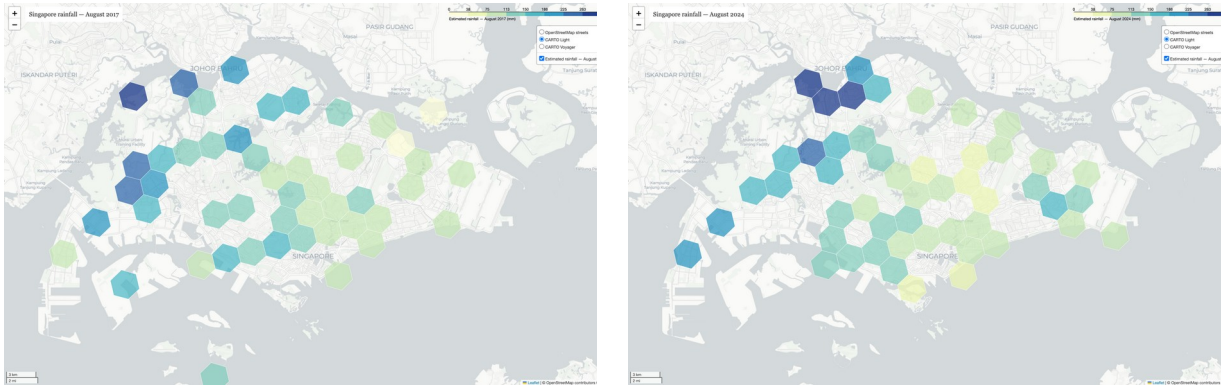


Figure 3. August rainfall in 2017 (left) and 2024 (right), shown with a common colour scale.

The unweighted cell means are similar—about 136 mm in August 2017 and 128 mm in August 2024—but the location of wetter cells changes. The 2017 maximum is about 299 mm and the 2024 maximum about 277 mm. This contrast is more informative than a single national average because it retains the spatial rearrangement of rainfall between years.

4.1.3 Long-period H3 pattern

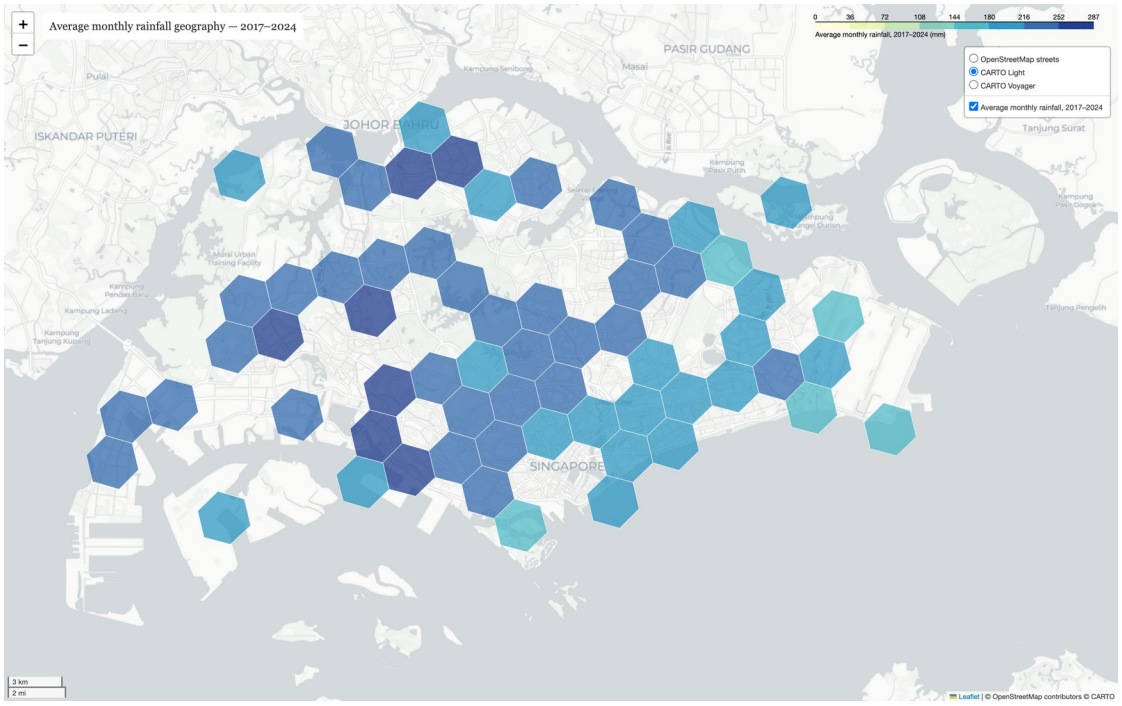


Figure 4. Coverage-normalised mean monthly rainfall across 2017–2024 in sensor-supported H3 resolution-7 cells.

4.1.4 Official planning-region summary

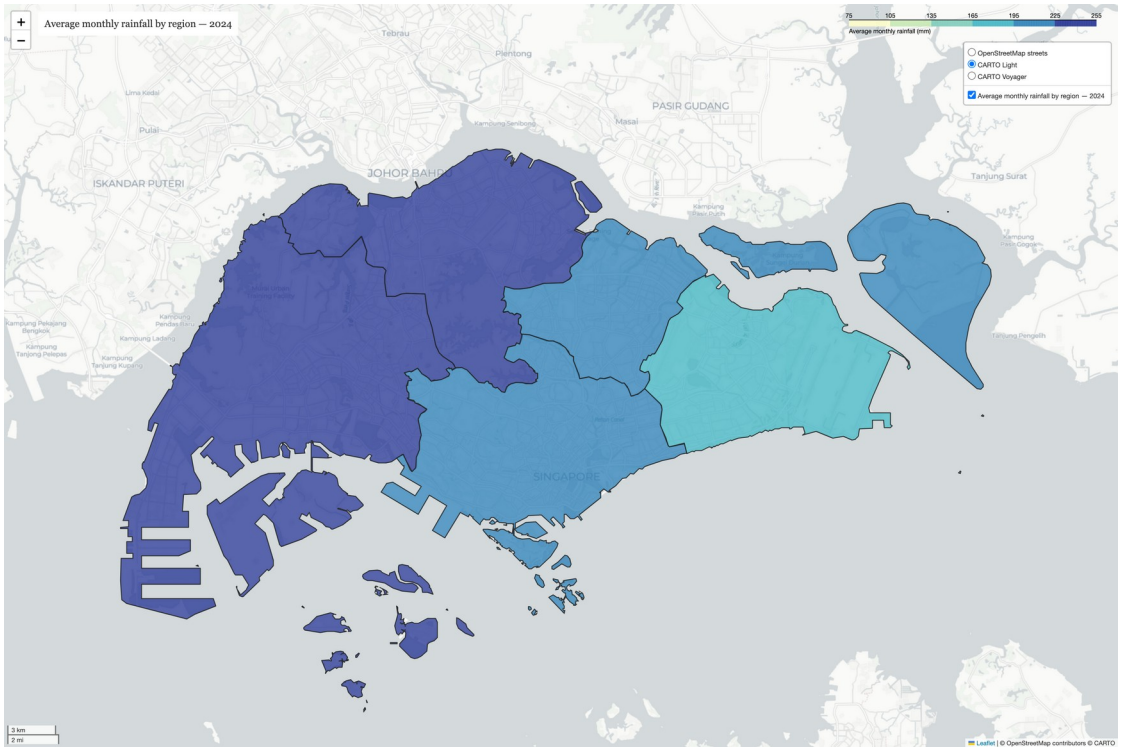


Figure 5. Mean monthly rainfall by official planning region in 2024. Source boundary: URA Master Plan 2019 Region Boundary (No Sea) GeoJSON.

Across the 63 retained H3 cells, the mean monthly value ranges from approximately 147 to 286 mm. Higher values occur frequently in the north and west, while several eastern and coastal cells fall toward the lower end of the scale. Because most cells contain one gauge, the polygons should be read as a compact display of station evidence rather than continuous areal estimates.

All five regions have twelve coverage-qualified months. North records the highest regional mean at about 254 mm per month, followed by West at 247 mm; East is lowest at 195 mm. North-East and Central lie between these values at approximately 225 and 214 mm. The number of active stations differs by region, so this equal-station summary describes the observed network and does not imply uniform rainfall within each region.

4.2 Interannual variation

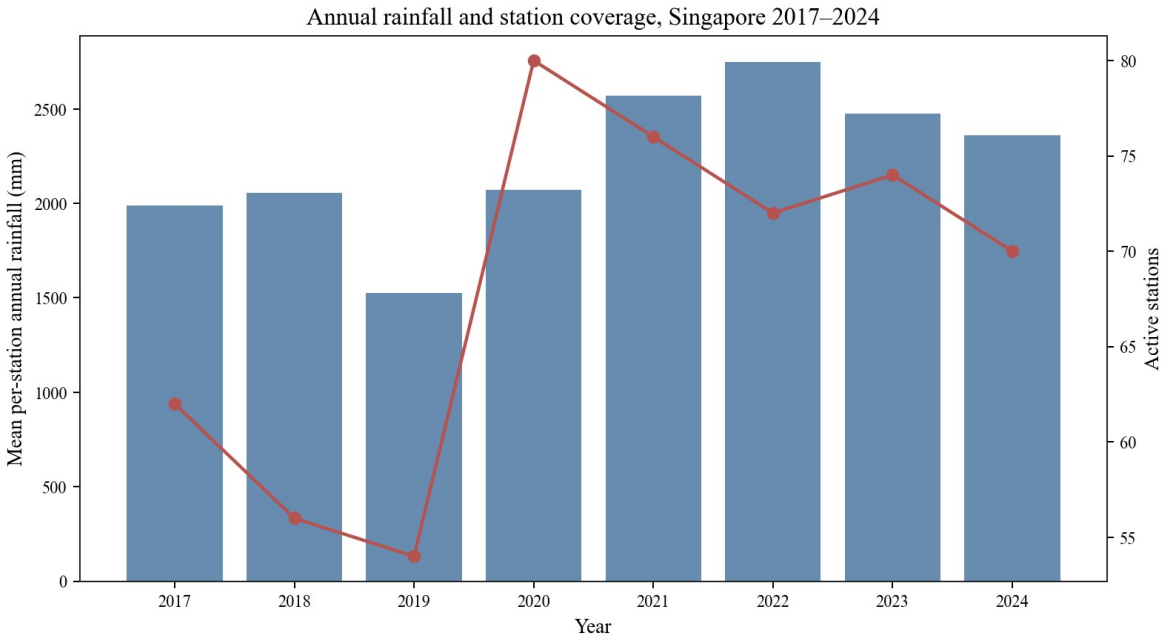


Figure 6. Mean annual rainfall per station with contributing-station count.

Mean annual rainfall per station ranges from 1,526 mm in 2019 to 2,750 mm in 2022, an 80% difference. The contributing network ranges from 54 to 80 stations, so a stable-station cohort and annual completeness rates are needed to test whether the ranking is robust.

4.3 Monthly seasonality

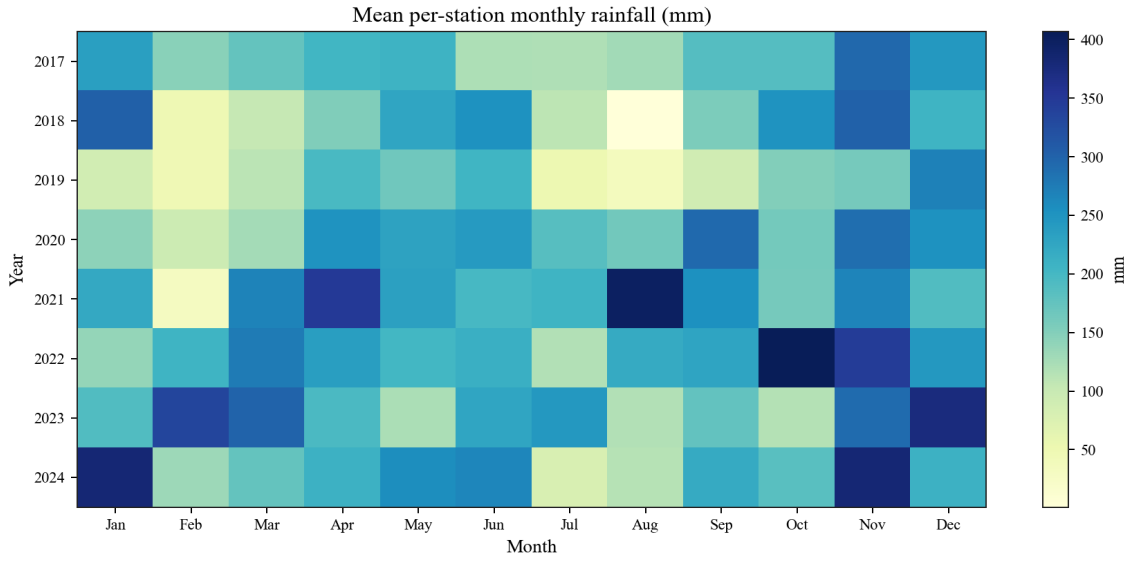


Figure 7. Year-by-month mean rainfall per station.

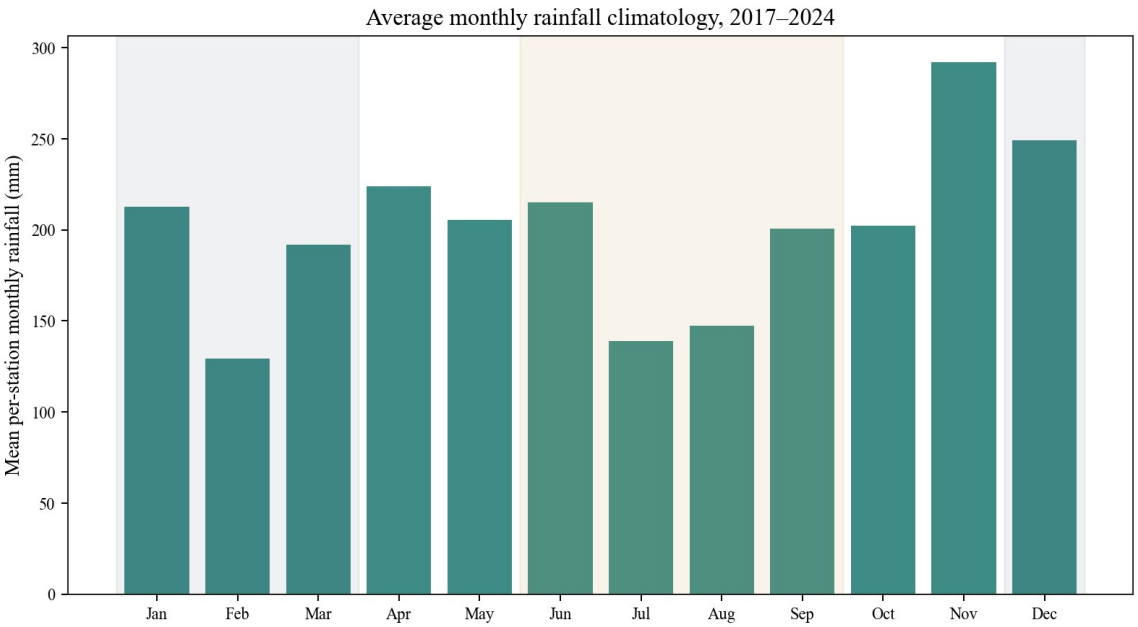


Figure 8. Average monthly rainfall pattern with monsoon context.

The monthly views show a recurring but variable seasonal cycle. Wetter Northeast Monsoon months and inter-monsoon peaks agree broadly with Singapore's documented climate, while individual years depart from the average [2]. The project data are observational and do not establish a causal meteorological mechanism.

4.4 Diurnal variation

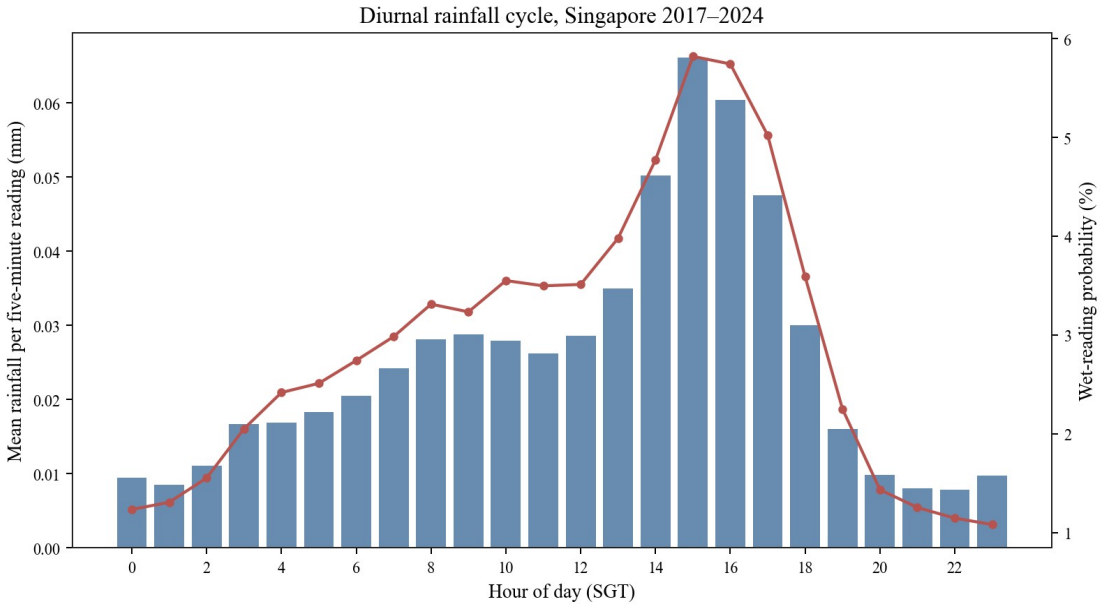


Figure 9. Diurnal rainfall intensity and wet-reading frequency.

Wet-reading frequency rises from about 1.1% at 23:00 to about 5.8% at 15:00 in the reported summary, a roughly 5.3-fold difference. The peak around 14:00–16:00 SGT is consistent with afternoon convection, while seasonal stratification tests whether the pattern persists under different monsoon regimes.

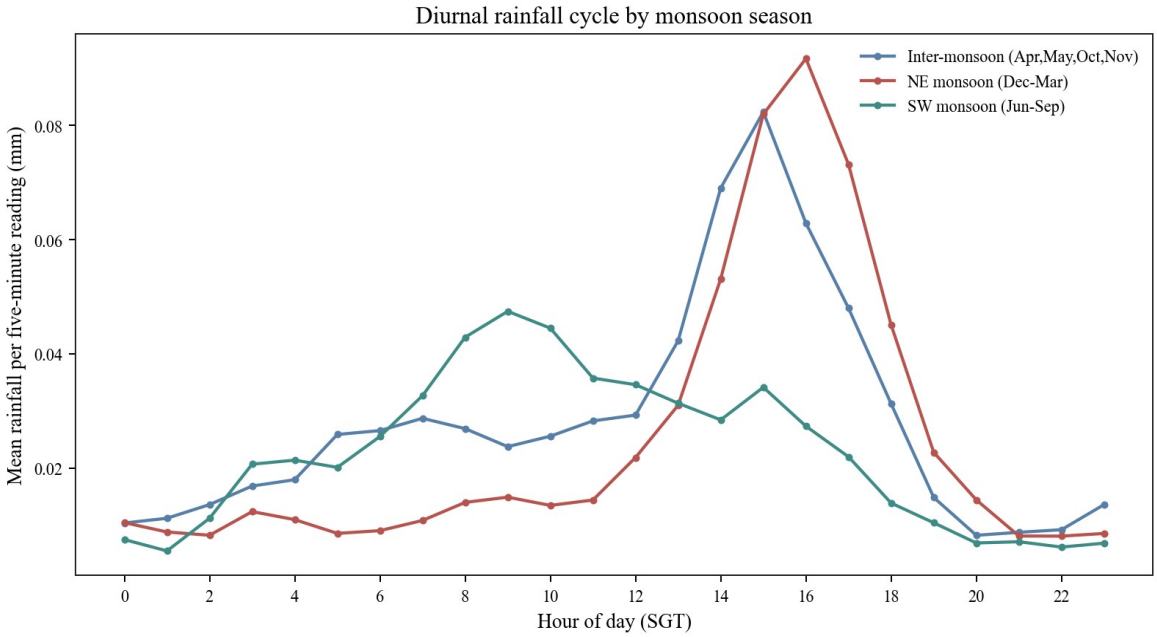


Figure 10. Diurnal rainfall pattern by monsoon season.

4.5 Heavy-rain station-days

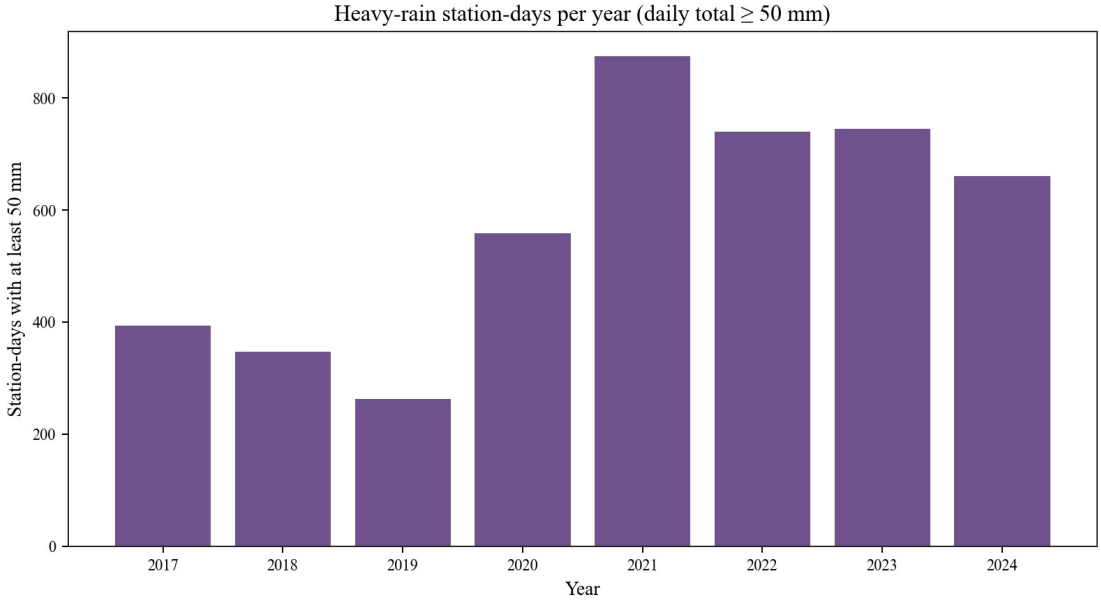


Figure 11. Count of station-days with at least 50 mm of rainfall.

The raw count is lowest in 2019 (263) and highest in 2021 (875). Because more operating stations create more opportunities to record an event, the preferred follow-up measure is heavy station-days per 1,000 eligible station-days under a stated completeness rule.

5. Task 2 — rainfall-probability prediction

5.1 Target and feature contract

For station s, prediction time t and threshold q, the binary target is 1 when the sum of rainfall readings at t+5, t+10, …, t+30 minutes exceeds q. The current reading at t is excluded from the target. Training examples are created for q ∈ {0.2, 1, 2, 5, 10} mm.

Table 3. Predictive feature families and their modelling roles.

Feature family	Features	Purpose
Recent rain	Current and previous reading; rolling 30/60/180-minute sums; minutes since rain	Represent persistence and storm state
Time	Hour/month sine and cosine; weekend; NE/SW monsoon indicators	Represent daily and seasonal cycles
Station	Station index, latitude, longitude	Represent location and station effects
Query	Threshold q in mm	Allow one fitted pipeline to score tested thresholds

Interpretation. The experiment validates five thresholds. A numeric threshold input does not establish accurate interpolation at unseen thresholds, and a tree model does not guarantee that predicted probability decreases monotonically as q increases.

5.2 Primary Spark MLlib pipeline

The fitted pipeline applies StringIndexer to station identity, VectorAssembler to the full feature set and Spark MLlib Gradient-Boosted Trees. The station index remains categorical, and maxBins is set above the number of stations. Figure A6 shows the shared transformation and fitting path; Appendix B provides one architecture diagram for every evaluated model.

Table 4. Gradient-Boosted Trees settings associated with the canonical saved-model record.

GBT parameter	Value	Reason / implication
maxIter	40	Boosting rounds
maxDepth	5	Limits tree complexity
stepSize	0.1	Shrinkage
maxBins	128	Supports at least 91 station categories
subsamplingRate	0.7	Row subsampling per tree
cacheNodeIds	true	Trades memory for repeated tree operations
seed	42	Fixed estimator randomness

5.3 Temporal evaluation and canonical run

The final saved model is trained on years 2017–2023 and evaluated on 2024. The table below reports the canonical result associated with the submitted pipeline. The separate model-comparison benchmark uses a smaller test sample and is reported independently.

Table 5. Canonical saved GBT evaluation record: 2017–2023 training and 2024 testing.

Canonical run	Train rows	2024 test rows	ROC-AUC	PR-AUC	Accuracy	Weighted F1
Saved GBT pipeline	7,997,496	31,872,380	0.924	0.584	0.986	0.984

The 0.984 F1 value is MLlib's support-weighted F1. With a reported training positive rate near 1.9%, it is dominated by the negative class and must not be read as positive-event F1. ROC-AUC and PR-AUC measure ranking; they do not prove that the predicted probabilities are calibrated.

5.4 Threshold-specific performance

Table 6. Threshold-specific ranking metrics for GBT and logistic regression at the five evaluated thresholds.

Threshold (mm)	GBT ROC	GBT PR	LogReg ROC	LogReg PR
0.2	0.904	0.660	0.879	0.557
1	0.905	0.577	0.874	0.500
2	0.895	0.505	0.865	0.441
5	0.898	0.398	0.857	0.346
10	0.905	0.302	0.861	0.253

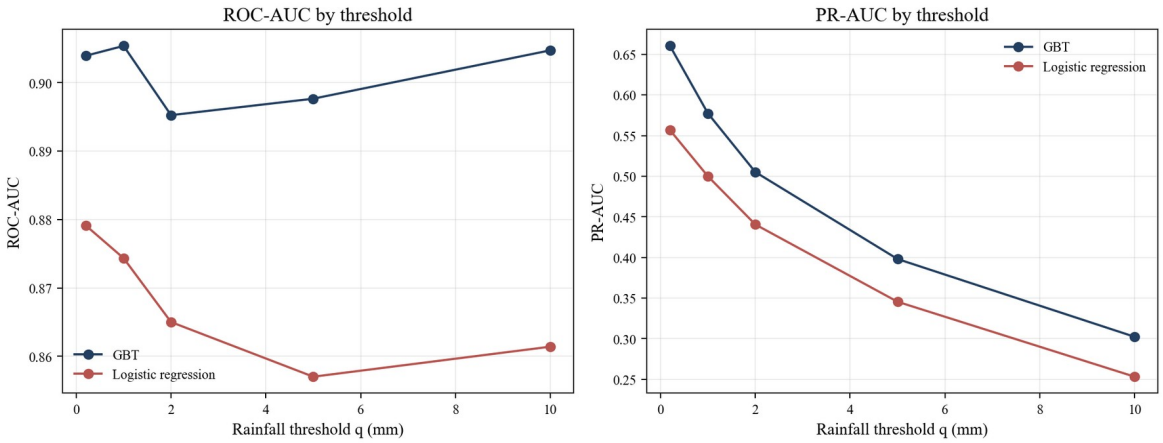


Figure 12. Threshold-specific ranking metrics reported for GBT and logistic regression.

GBT outperforms logistic regression at all five reported thresholds. GBT PR-AUC falls from 0.660 at 0.2 mm to 0.302 at 10 mm, consistent with the increasing rarity and difficulty of higher-accumulation events. Positive prevalence by threshold is required to quantify the baseline level of each PR curve.

5.5 Limits of the retained evaluation

- Positive-class evidence unavailable: per-threshold prevalence, validation-selected cutoffs, confusion matrices, precision, recall and positive-class F1 are not retained.
- Calibration evidence unavailable: the report has no Brier score, log loss or reliability diagram based on a separate validation year.
- Baseline evidence unavailable: persistence and prevalence-only probability baselines were not evaluated in the retained results.
- Threshold-generalisation evidence unavailable: intermediate thresholds and monotonicity violations across q were not evaluated.

6. Model and compute experiments

6.1 Model comparison

The model-comparison benchmark is a separate experiment. The six MLlib models share approximately 8.0 million training rows and 1.59 million test rows. The GPU models use about 401,000 rows per split, different sequence features and a different evaluator; their timing and quality therefore describe practical implementations rather than a controlled algorithm comparison. The model structures are documented independently in Figures M1–M8 so that algorithmic differences are separated from performance claims.

Table 7. Model-comparison results; GPU rows use different samples, features and evaluators from MLlib rows.

Model	Runtime	Train (s)	Predict (s)	ROC-AUC	PR-AUC
GBT	MLlib CPU	278.6	8.16	0.925	0.586
Random Forest	MLlib CPU	173.5	8.43	0.902	0.560
Logistic Regression	MLlib CPU	27.9	4.91	0.897	0.436
Linear SVC	MLlib CPU	44.3	5.44	0.875	0.468
MLP	MLlib CPU	321.1	7.95	0.921	0.557
Factorization Machines	MLlib CPU	278.3	5.51	0.814	0.196
LSTM	GPU	75.3	0.44	0.915	0.476
TCN	GPU	79.9	0.36	0.915	0.473

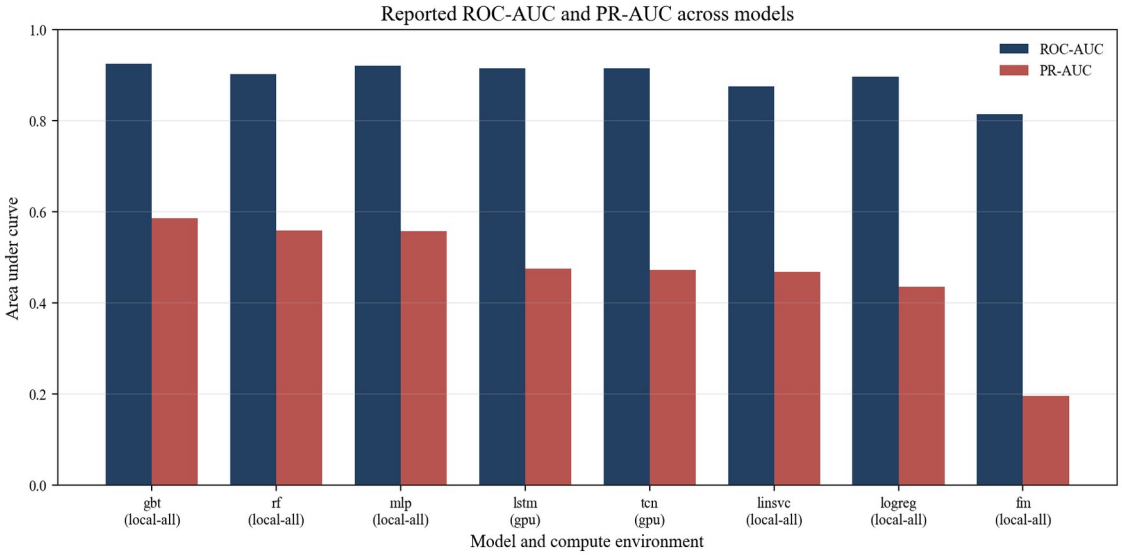


Figure 13. Reported ROC-AUC and PR-AUC across model implementations.

GBT has the highest reported PR-AUC in this experiment. Random Forest and MLP are competitive, while the linear models provide faster baselines. The short GPU inference times are not directly comparable with MLlib because the scored row counts, feature representations and frameworks differ.

6.2 Deployment comparison

Four MLlib models were run across five Spark configurations on approximately one million sampled training rows. The sample counts differ slightly by configuration, and the configurations also change

cores, heap budgets and shuffle partitions. The timings are therefore descriptive observations rather than a controlled estimate of deployment overhead. Figure A7 specifies the matched-resource protocol needed for a causal deployment comparison.

Table 8. Deployment observations; sampled rows and resource budgets are not matched across configurations.

Configuration	GBT train rows	GBT (s)	RF (s)	LogReg (s)	LinSVC (s)
local-all	999,587	64.0	42.4	13.4	27.0
standalone-2x2	1,000,314	108.9	75.6	23.5	41.5
standalone-4x2	1,000,771	86.2	51.2	19.9	36.6
docker-2w	1,000,314	101.8	71.7	21.6	46.2
docker-3w	1,000,771	82.4	59.4	20.2	39.1

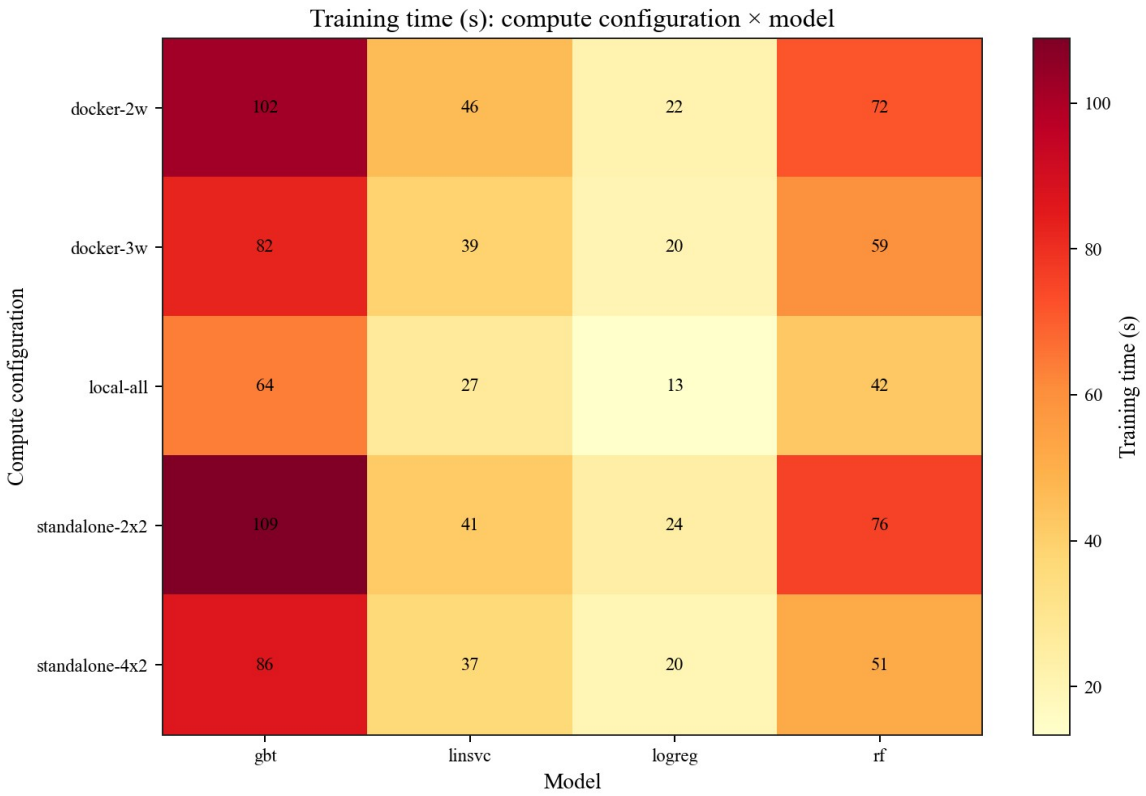


Figure 14. Training time by deployment configuration; resources are not held constant.

Local execution is fastest in the reported table. Within each cluster family, more allocated cores reduce GBT training time. These results are compatible with executor, serialization and container overhead on a shared host, but the current design cannot isolate those causes. Quality metrics remain similar because deployment changes execution, not the learning objective.

6.3 Protocol for future controlled systems experiments

Table 9. Proposed controlled systems experiments; these protocols are not reported as executed results.

Experiment	Controlled design	Report output
Matched deployment overhead	Same frozen row IDs, four cores, memory budget, partitions and GBT; local[4] vs 2×2 Standalone vs 2×2 Docker	Median and range over ≥3 measured runs; stage time, shuffle, spill, GC
Parallel scaling	Fixed data/model; local[1/2/4/8], then executor layouts with	Speedup S(p), efficiency E(p), through-

Experiment	Controlled design	Report output
	fixed total cores	put and p95 task time
Feature reuse break-even	Recompute features per fit versus materialise once and reuse for 1, 3 and 6 fits	End-to-end elapsed time and measured break-even number of fits

7. Threats to validity and proposed extensions

The following threats have the greatest potential to affect target construction, feature timing or interpretation. Their effect on the reported metrics cannot be established from the retained artifacts alone.

Table 10. Retained implementation risks and the validity checks needed before stronger predictive claims.

Priority	Risk	Implication / required check
P0	Temporal window boundaries	Confirm that rolling 30 minutes contains the intended six five-minute totals; rangeBetween(-1800,0) can include seven grid points.
P0	Cadence integrity	A previous row is not necessarily t−5 minutes. Verify exact offsets for lag sequences and future labels when observations are missing.
P0	Temporal split boundary	Ensure training rows near year-end do not use 2024 observations in their target windows.
P0	Training/inference parity	Training can search up to 24 h for last rain; inference currently loads six hours. Compare offline and online features for identical queries.
P1	Run traceability	Assign immutable run IDs to data, feature version, sample membership, model, metrics and figures.
P1	Probability validation	Add calibration, positive-class metrics and unseen-threshold tests before operational claims.

Interpretation. These are implementation risks rather than evidence that the reported ranking metrics are incorrect. Any future correction that changes features or labels requires a new versioned run and a jointly regenerated model, metric set and figure set.

7.1 Proposed temporal-partitioning experiment

A strong extension is to compare the corrected station-wide temporal window with station-month blocks that include boundary “halo” data. Each monthly block must include at least 180 minutes of prior context, 30 minutes of future context and sufficient last-wet state, then emit only rows owned by its core month. The optimised output must match the reference output exactly before runtime is compared.

- Correctness: identical keys, feature values and labels, including month/year boundaries and missing-slot cases.
- Systems evidence: runtime, shuffle read/write, spill, task-duration distribution and executor utilisation.
- Honest outcome: overlap may cost more than added parallelism; no speed-up is still a useful finding when explained.

8. Limitations and conclusion

The analysis is limited by changing station coverage, incomplete rule-level cleaning counts, a project-selected high-reading cutoff, sampled training, unresolved probability calibration and single-host compute experiments. Deep-model comparisons use smaller datasets and different feature representations. The saved model’s evaluator reports weighted rather than positive-class F1.

Within those limits, the project demonstrates a coherent Spark workflow from large raw CSV files to reusable Parquet, distributed temporal processing, MLlib training and report-ready summaries. The weather record is marked by the wet 2021–2022 period, dry 2019, an afternoon wet-reading peak and clear spatial heterogeneity. GBT provides the strongest retained predictive result through its PR-AUC advantage

over the logistic baseline at each tested threshold. On the shared laptop, the additional executor and container layers introduce enough overhead that they do not offset the smaller per-process resource allocation; this observation is specific to the tested host.

9. AI usage declaration

OpenAI ChatGPT and Codex were used to support idea development, language editing, document formatting, analysis planning, and preparation of diagrams and illustrations. Responsibility for the submitted analysis, interpretation and quality remains with the project team.

References

[1] National Environment Agency. Historical Rainfall across Singapore datasets, 2017–2024. data.gov.sg. Source link

[2] Meteorological Service Singapore. Climate of Singapore. Source link

[3] Apache Spark. MLlib classification and regression documentation, Spark 3.5.3. Source link

[4] Apache Spark. BinaryClassificationMetrics API, Spark 3.5.3. Source link

[5] Apache Spark. Monitoring and instrumentation, Spark 3.5 series. Source link

[6] Apache Spark. SQL performance tuning and Adaptive Query Execution, Spark 3.5 series. Source link

[7] AI assistance statement. OpenAI ChatGPT and Codex were used to support idea development, language editing, document formatting, analysis planning, and preparation of diagrams and illustrations. Responsibility for the submitted analysis, interpretation and quality remains with the project team.

Appendix A. Distributed systems architecture

This appendix makes the execution boundaries explicit. Solid arrows show primary data or task flow; dashed arrows show scheduling, shuffle, fitting or instrumentation. The Standalone and Docker layouts remain within one physical laptop and therefore demonstrate process-level distribution rather than multi-machine scale-out.

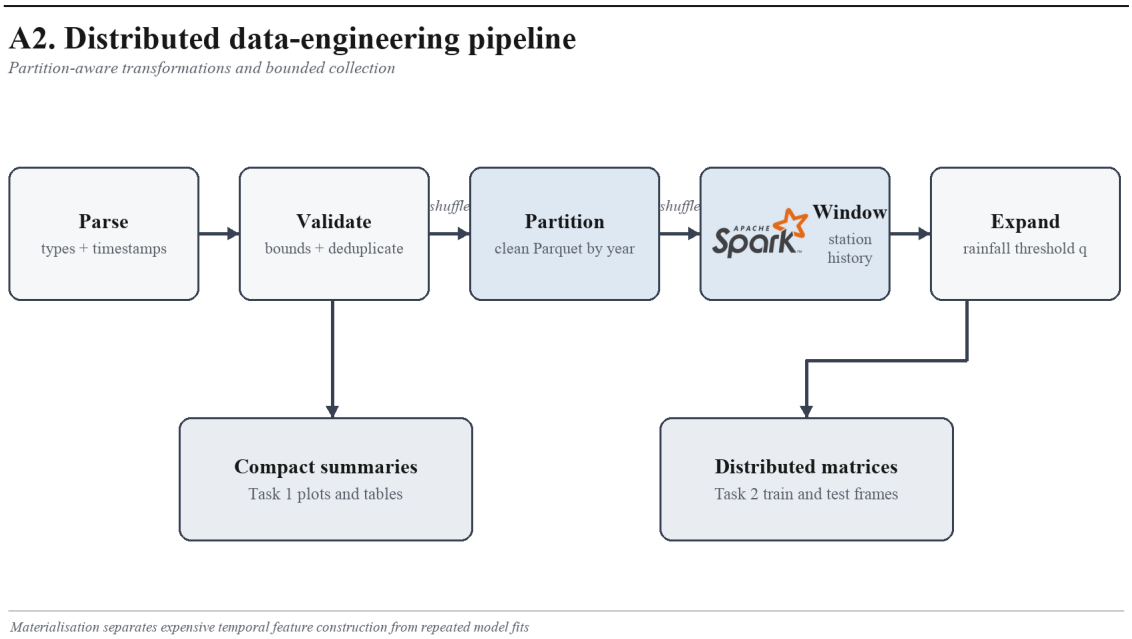


Figure A2. Distributed data-engineering path, including partitioning, shuffle boundaries and feature materialisation.

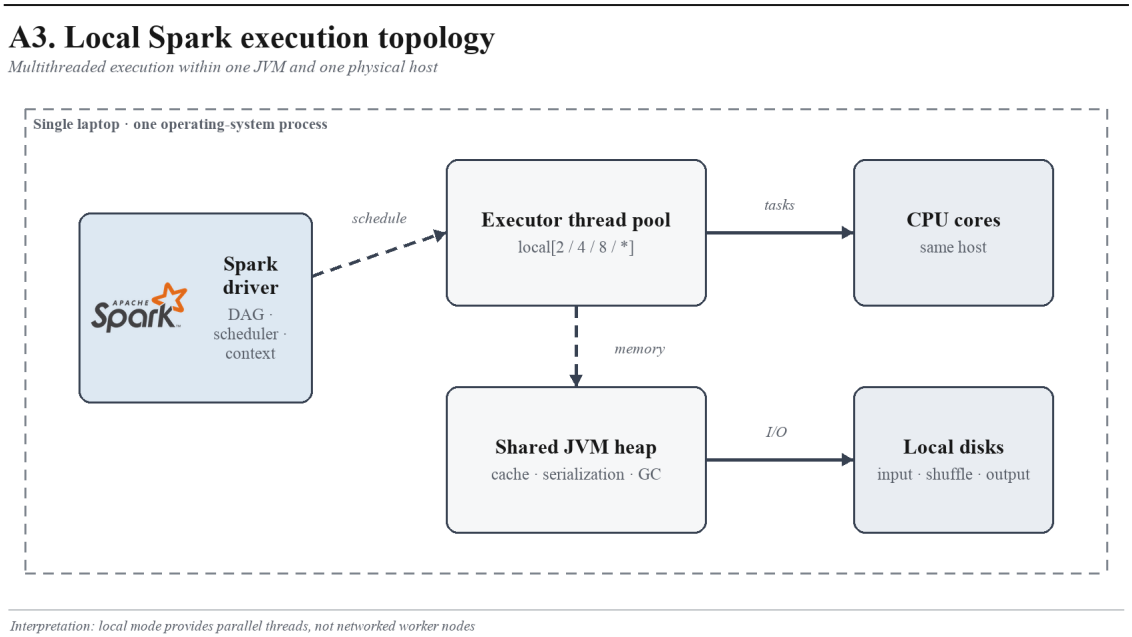


Figure A3. Local Spark uses executor threads inside the driver JVM and shares one host resource pool.

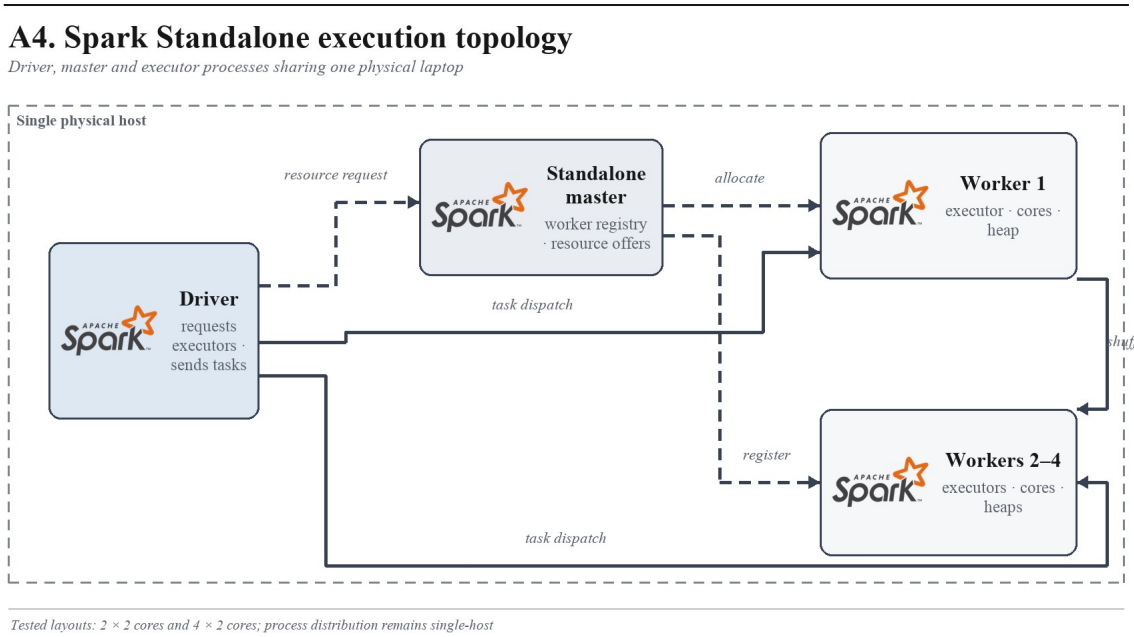


Figure A4. Spark Standalone separates the master, driver and executor JVMs while retaining a shared physical-host boundary.

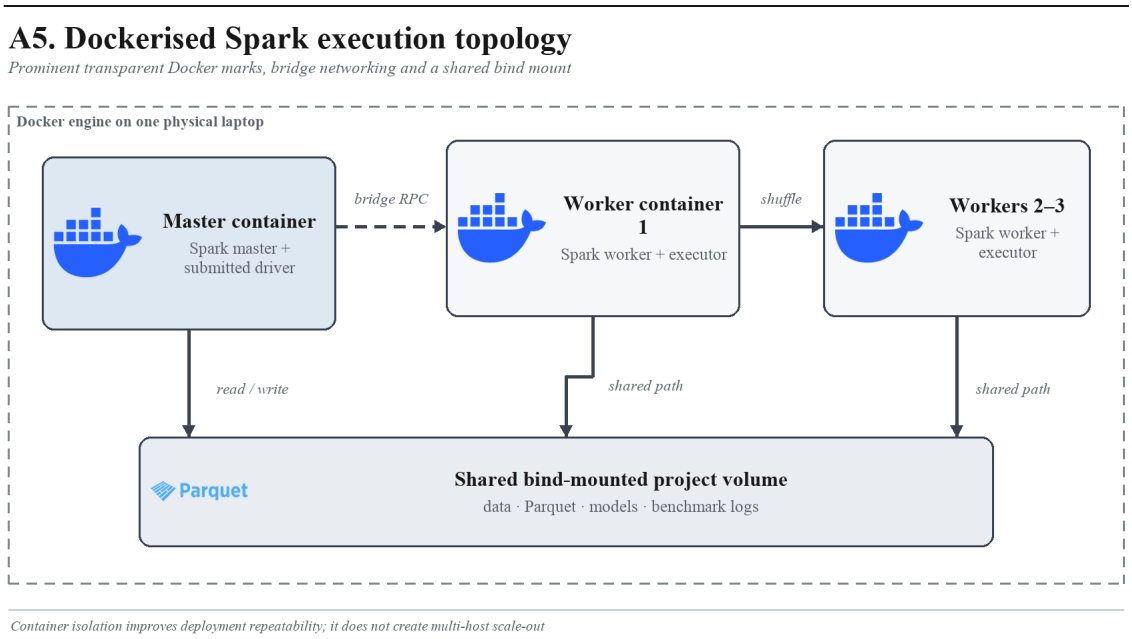
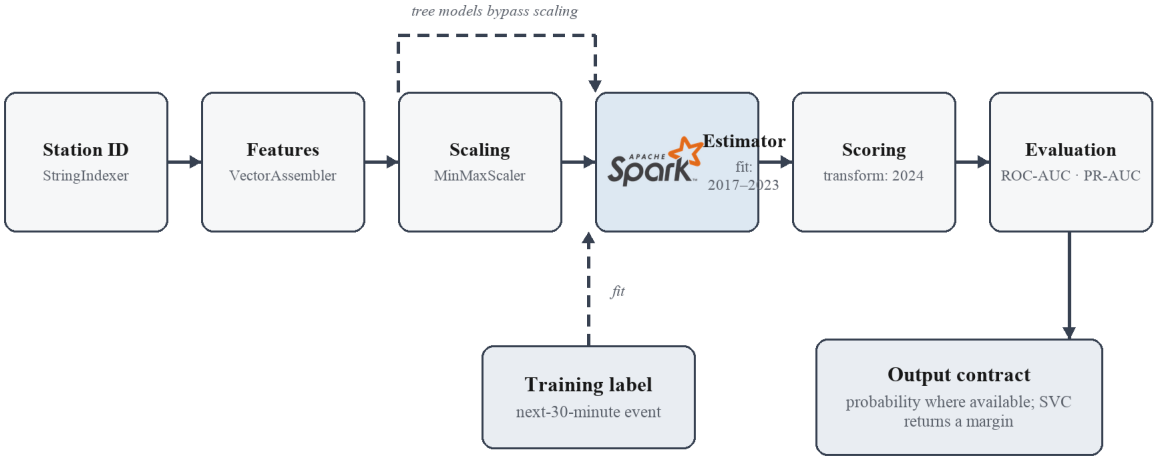


Figure A5. Docker Spark isolates the master and workers as containers connected by a bridge network and shared bind mount.

A6. Common Spark MLlib pipeline

Readable estimator flow with optional scaling and an explicit fit path

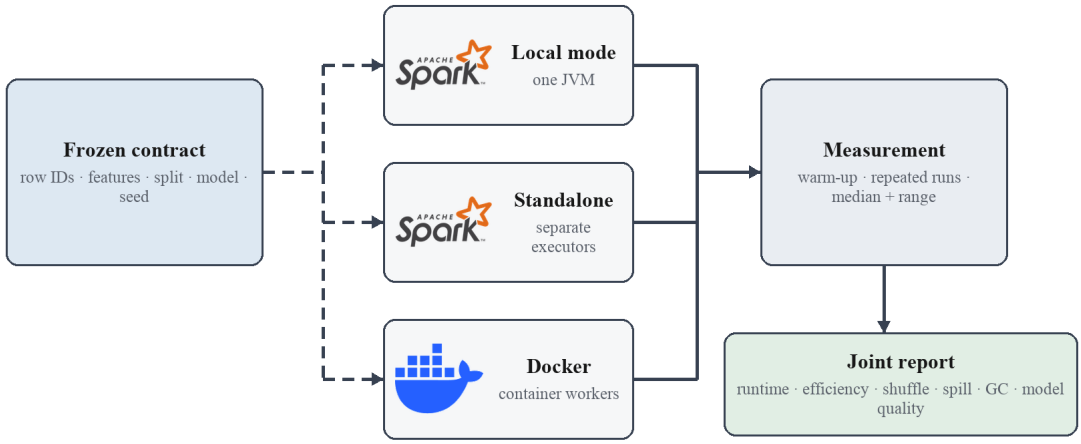


MLlib is represented by the Apache Spark mark because it has no separate official logo

Figure A6. Common MLlib transformation, fitting and evaluation path used by the six Spark classifiers.

A7. Controlled experiment design

A matched protocol for credible deployment and scaling comparisons



The same data, resources, partitions and evaluation set are required for causal comparison

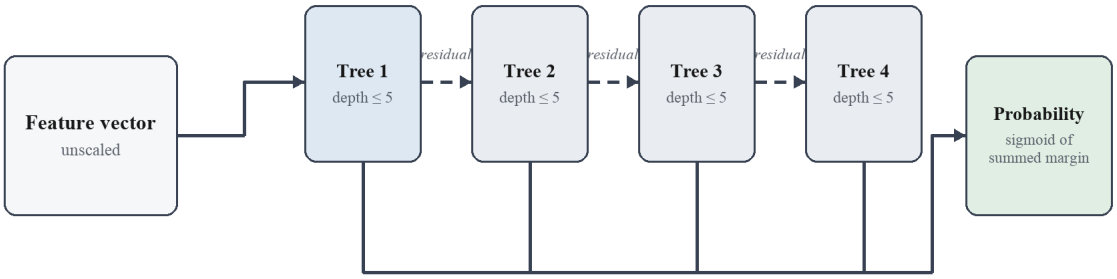
Figure A7. Proposed matched-resource protocol for comparing local, Standalone and Docker execution.

Appendix B. Model architecture atlas

Each evaluated model is shown separately because its computational assumptions differ. These figures document the implemented structure and experimental role; they do not substitute for the common evaluation tables in Section 6.

M1. Gradient-Boosted Trees

Sequential residual correction in an additive tree ensemble



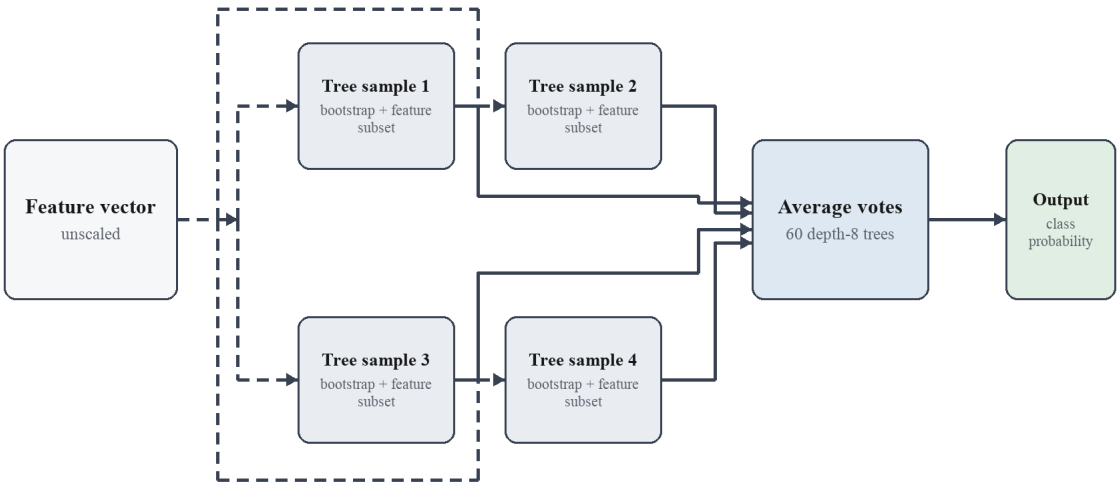
40 rounds · learning rate 0.1 · subsampling 0.70

Primary non-linear Spark MLlib model

Figure M1. Gradient-Boosted Trees: sequential residual fitting and additive probability score.

M2. Random Forest

Parallel bagging, feature subsampling and probability averaging

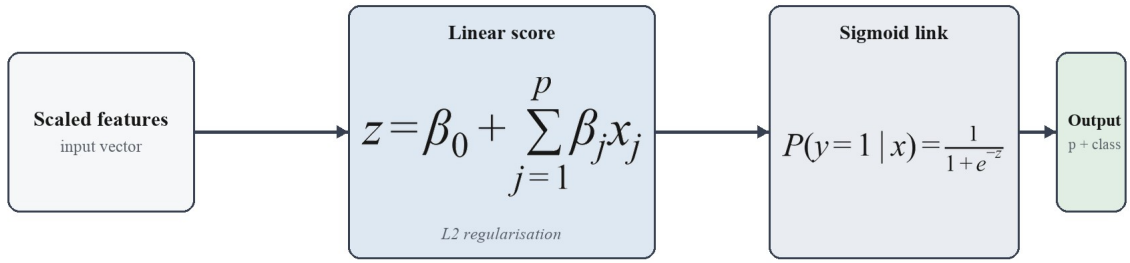


Bagged-tree comparison model

Figure M2. Random Forest: independent bagged trees followed by vote or probability averaging.

M3. Logistic Regression

Regularised linear score with a calibrated link function

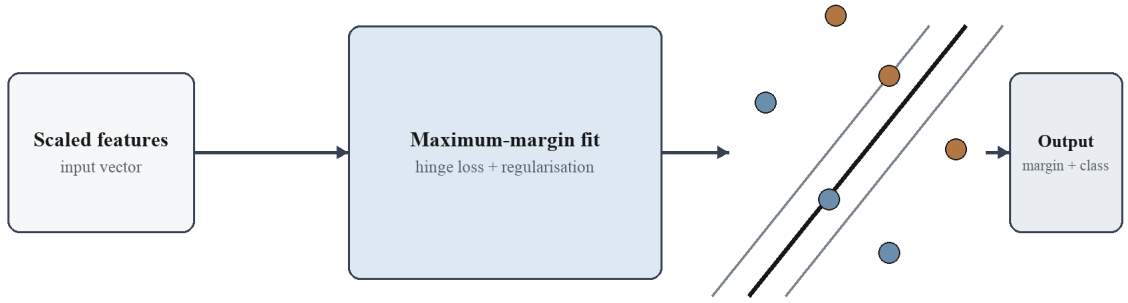


Subscripts and superscripts are typeset mathematically to avoid missing-glyph boxes

Figure M3. Logistic regression: scaled linear score with L2 regularisation and sigmoid probability.

M4. Linear Support Vector Classifier

Maximum-margin separation with an explicit decision score

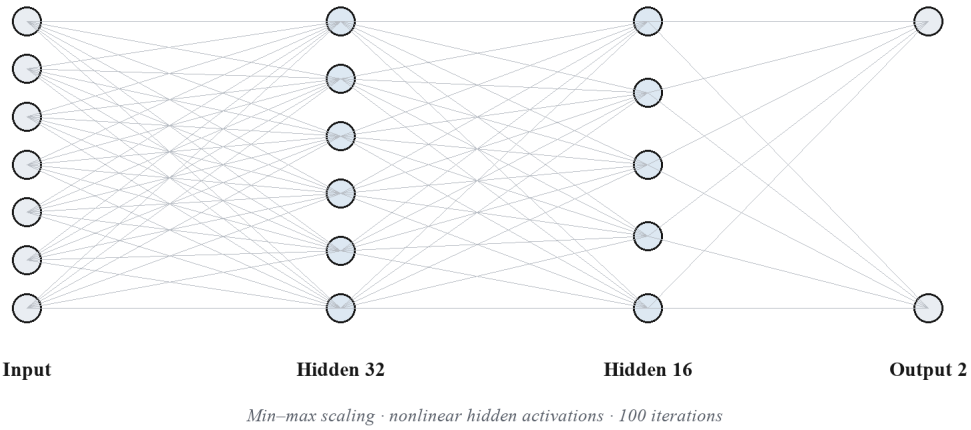


Linear SVC does not provide a native probability without a separate calibration step

Figure M4. Linear SVC: scaled maximum-margin classifier that yields a ranking margin rather than a native probability.

M5. Spark MLlib multilayer perceptron

Feed-forward architecture: input → 32 → 16 → 2

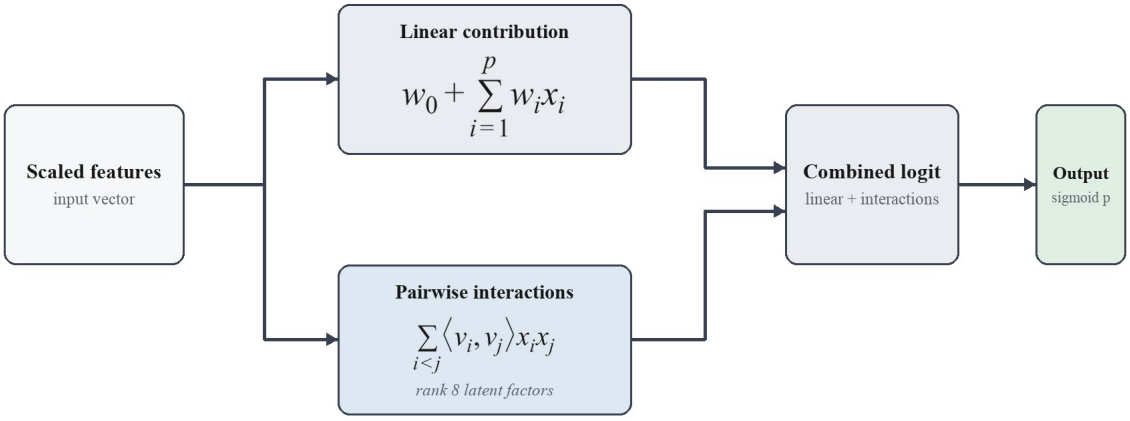


Spark MLlib neural baseline

Figure M5. Spark MLlib multilayer perceptron with 32- and 16-unit hidden layers.

M6. Factorization Machines

Linear effects combined with learned low-rank interactions

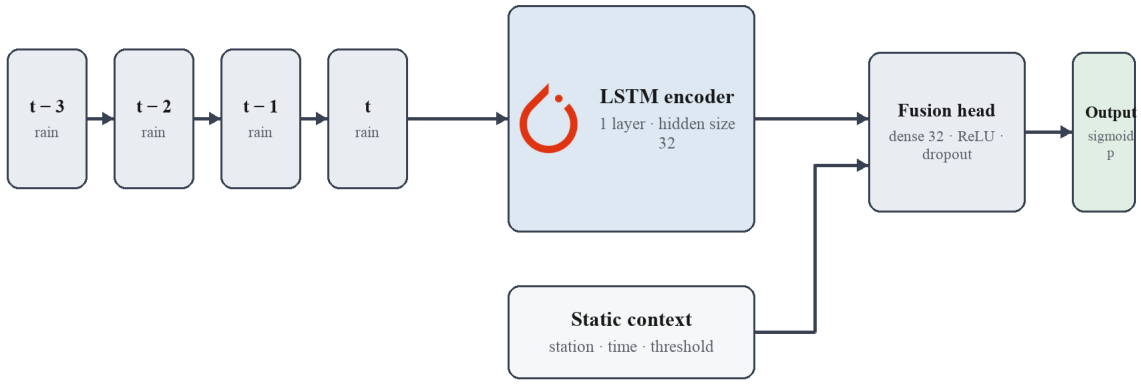


Interaction-aware model with properly typeset indices

Figure M6. Factorization Machines combine linear effects with learned rank-8 pairwise interactions.

M7. LSTM sequence model

Recent rainfall dynamics fused with static context on one device

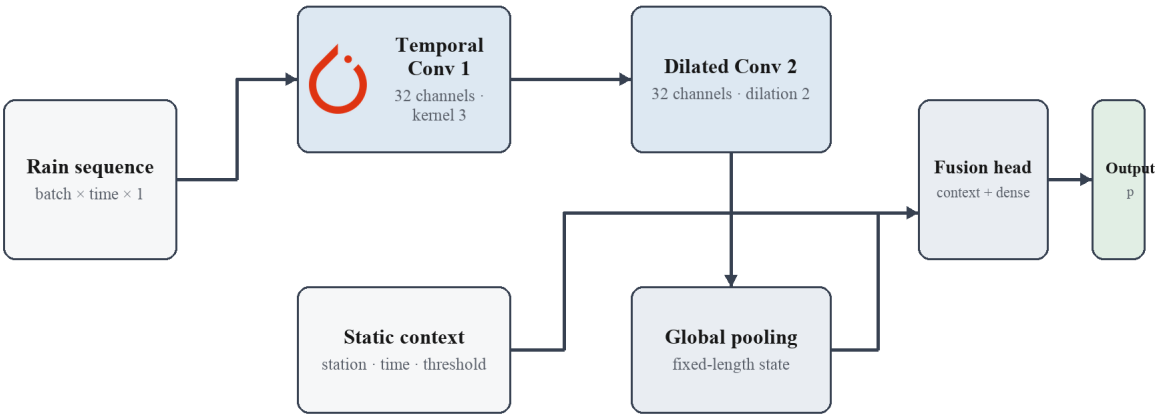


Supplementary single-device PyTorch experiment; not distributed Spark training

Figure M7. LSTM sequence model with static-context fusion and a dense PyTorch classification head.

M8. Temporal convolutional network

Dilated temporal filters, global pooling and context fusion



Supplementary single-device PyTorch experiment

Figure M8. Temporal CNN with standard and dilated convolutions, global pooling and static-context fusion.